

ANALISIS SPEEDUP SORTING PARALEL: MERGE SORT DAN BITONIC SORT MENGUNAKAN OPENMP

GHEA VICTORIA NEISCA (2430305030010)
MEYLIN SAZKIA PUTRI (2430305030004)
ELGA EDENIA HAFITZ (2430305030014)
HELEN EDENIA HAFITZ (2430305030034)

KELOMPOK 2

Septian Geges, S.KOM,. M.KOM

KOMPUTASI PARALEL DAN TERDISTRIBUSI



Agenda

- Latar Belakang & Masalah
- Tujuan & Batasan
- Desain Arsitektur
- Implementasi
- Hasil & Analisis Performa
- Pembahasan
- Kesimpulan

Mengapa Sorting Paralel?

- Data terus bertumbuh setiap tahun
- Sorting 1 juta elemen butuh ~350ms
- Processor modern punya 4-16 core
- Core sering tidak dimanfaatkan optimal
- Bagaimana memaksimalkan semua core?

Tujuan & Batasan

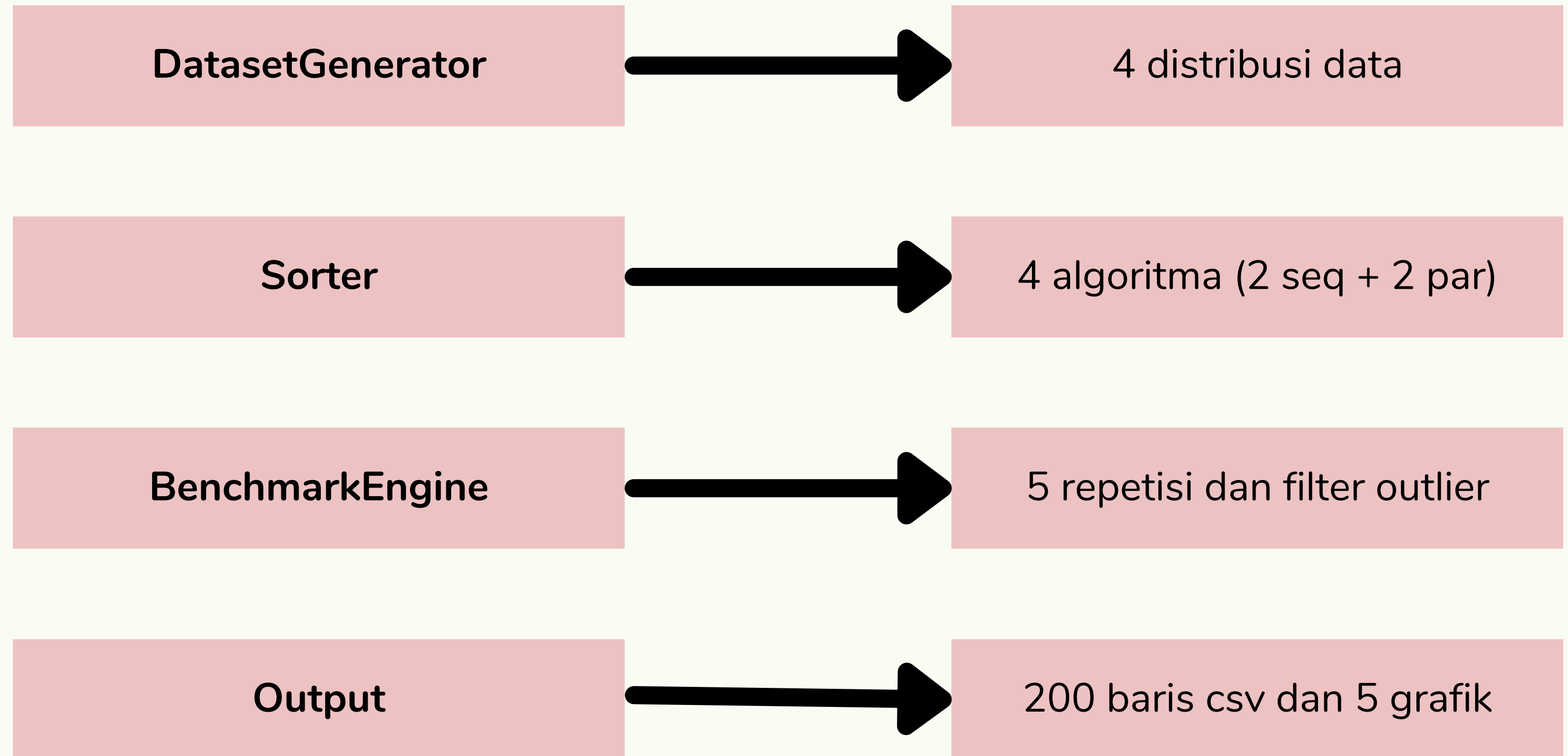
Tujuan

- Implementasi Merge Sort & Bitonic Sort paralel
- Ukur speedup, efisiensi, scalability
- Validasi hasil dengan Amdahl's Law
- Temukan konfigurasi thread optimal

Batasan

- Shared-memory OpenMP, bukan MPI
- Dataset tetap 1 juta elemen integer
- Maksimum 8 thread, 1 mesin

Desain Arsitektur (Pipeline Sistem)



Implementasi Merge Sort

```
#pragma omp parallel
{
    #pragma omp single nowait
    {
        mergeSortPar(data, 0, n-1);
    }
}
```

Entry Point Paralel

```
if (size <= cutoff_) {
    MergeSortSeq().sort(sub);
    return;
}
#pragma omp task untied shared(arr)
{ mergeSortPar(arr, left, mid); }
#pragma omp task untied shared(arr)
{ mergeSortPar(arr, mid+1, right); }
#pragma omp taskwait
merge(arr, left, mid, right);
```

Inti Paralelisme

Implementasi Bitonic Sort

```
if (stageSize < cutoff_) continue
#pragma omp parallel for schedule(static)
for (i=0; i<n; i++) {
    compareAndSwap(data, i, i^j)
}
// implicit barrier di sini
```

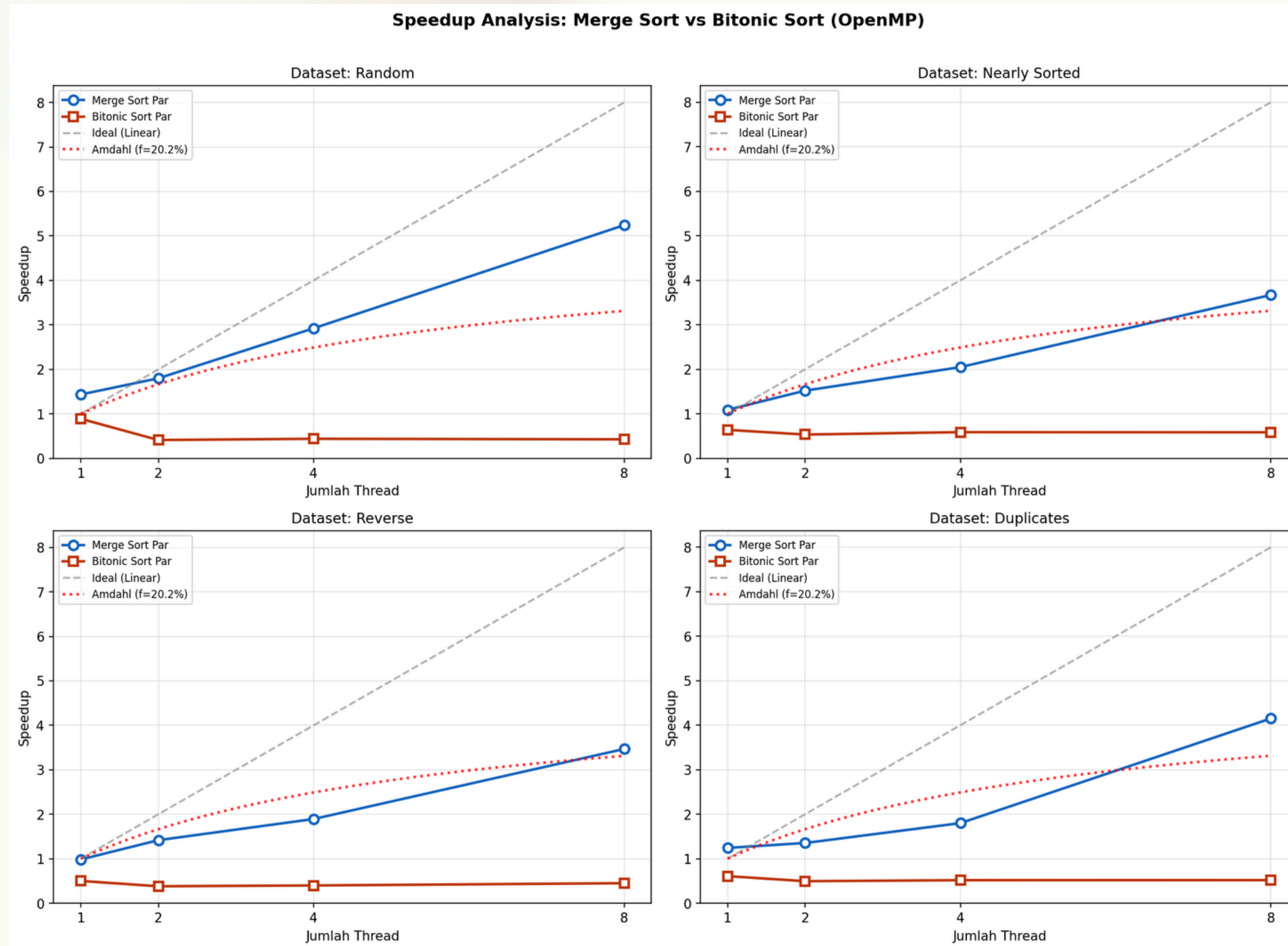
Poin kunci:

- Pola XOR distribusi pasangan simetris
- 45 barrier untuk $n=1.048.576$ elemen
- Barrier = semua thread tunggu bersama

Implementasi: Cutoff & Fault Tolerance

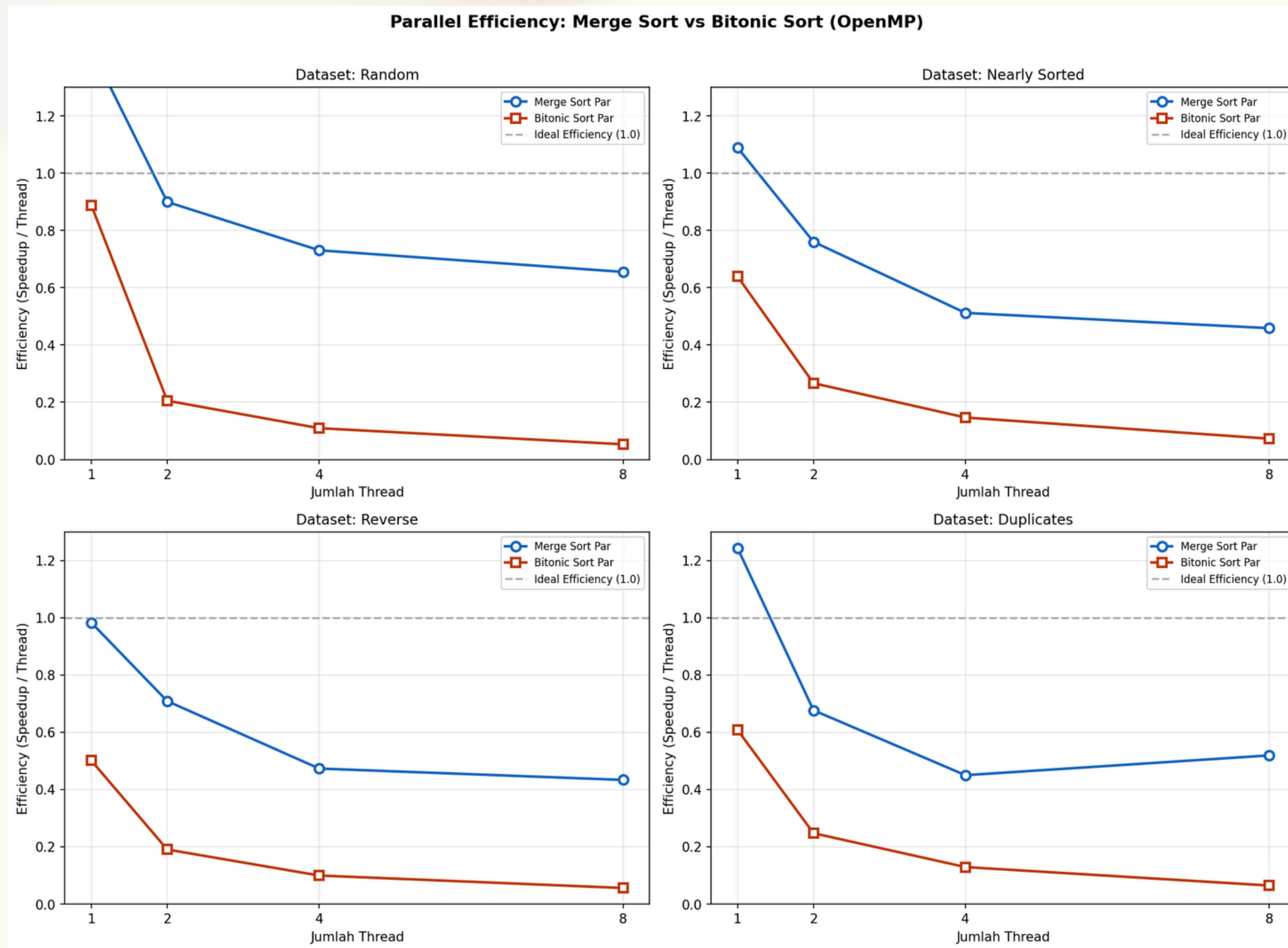
Cutoff	Fault Tolerance
• Stage kecil → kerjakan sequential • Overhead thread > manfaat paralel • Uji 3 nilai: 512, 1024, 2048	• withRetry() → coba ulang 2x jika error • Backoff: tunggu 100ms lalu 200ms • validateSorted() → catat PASS/FAIL

Hasil: Speedup Merge Sort



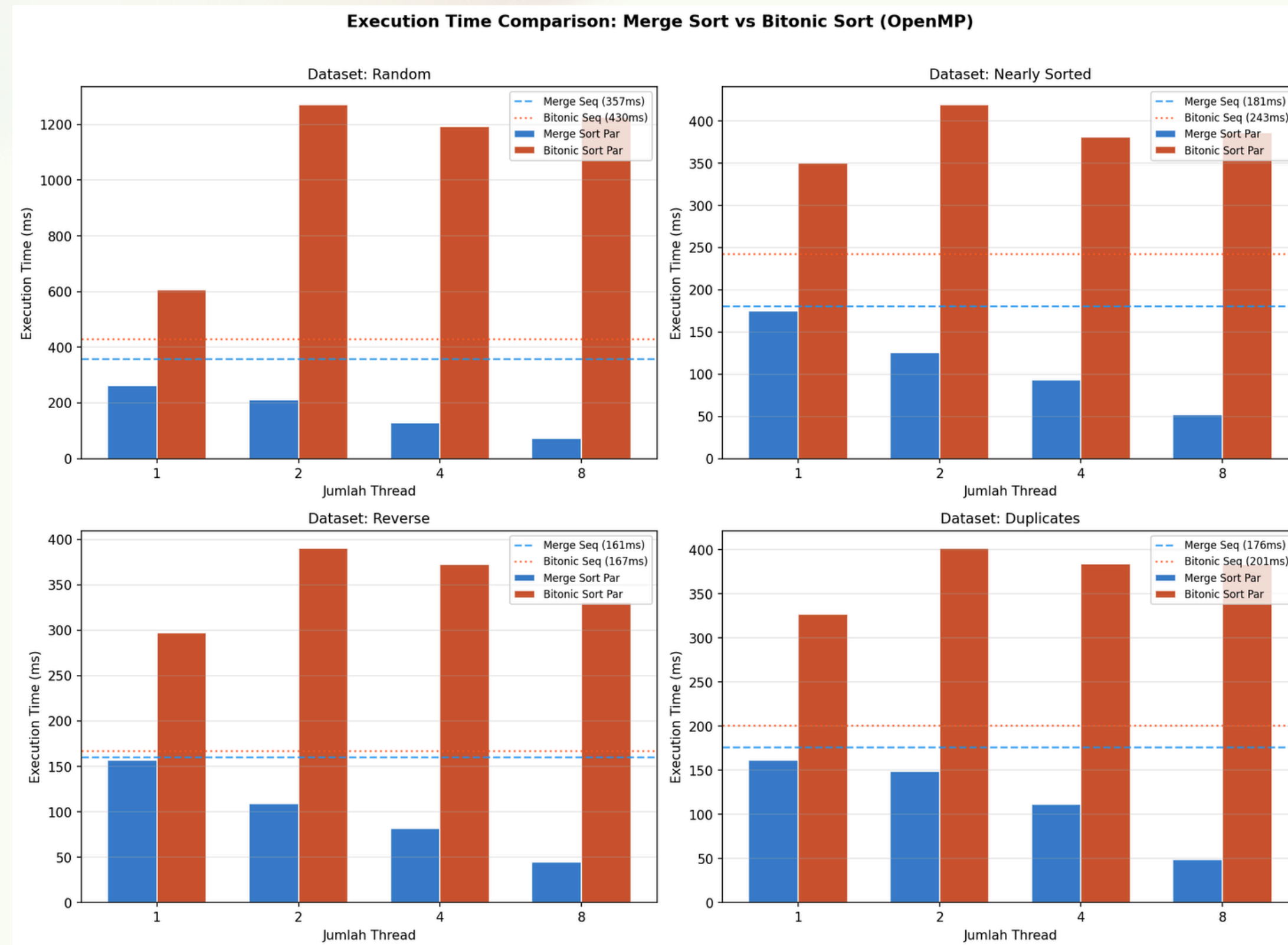
- Merge Sort naik dari 1x → 5.03x
- Kurva Amdahl sangat mendekati empiris
- Bitonic Sort flat di bawah 1.0

Efisiensi Pararel per Dataset



- Merge Sort efisiensi turun bertahap
- Superlinear speedup terdeteksi (>1.0)
- Bitonic Sort mendekati nol

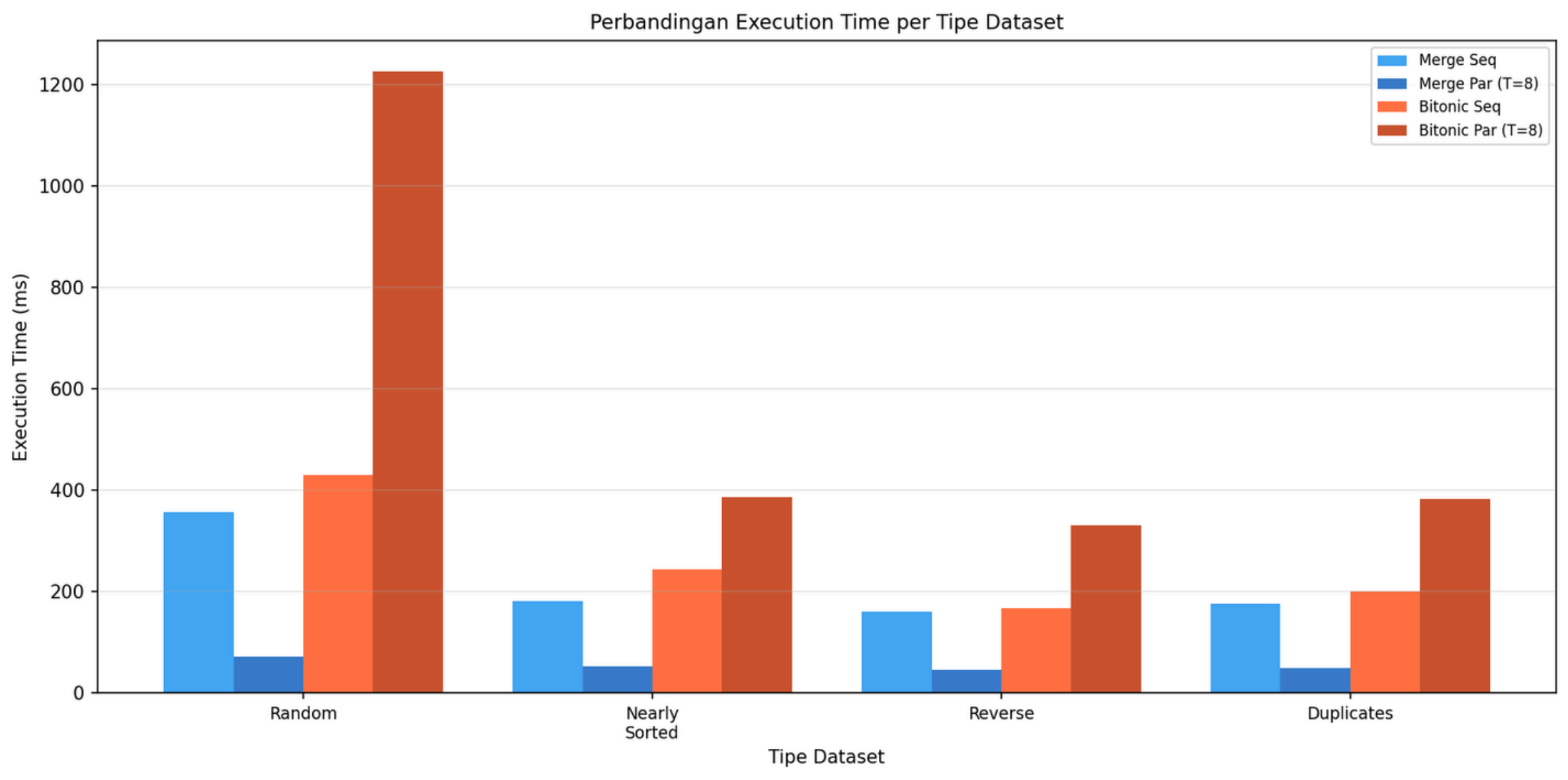
Execution Time: Sequential vs Parallel



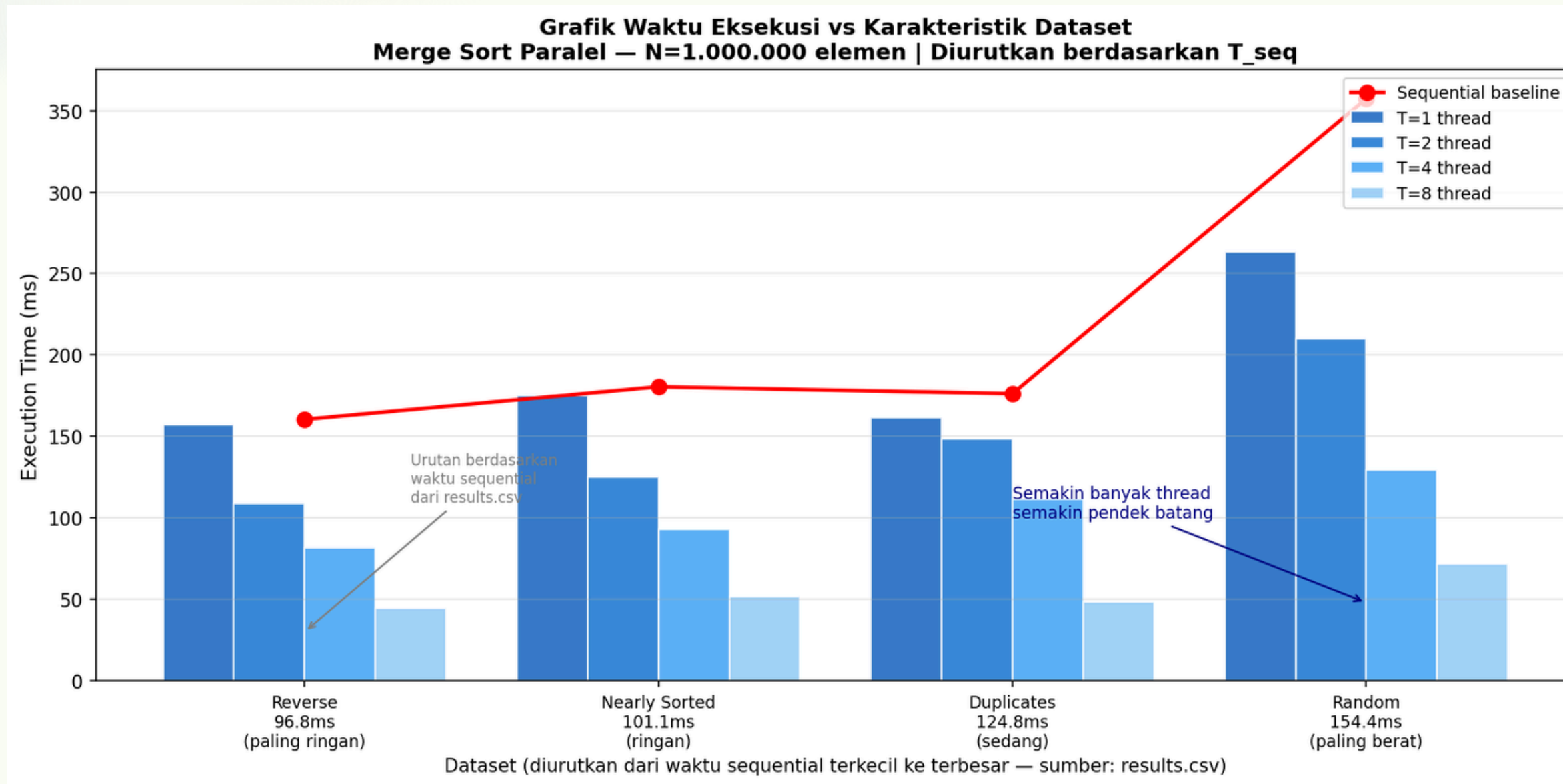
- Merge Sort (biru) makin pendek ✓
- Bitonic Sort (merah) selalu di atas ✗
- Konsisten di semua 4 dataset

Analisis Amdahl's Law

DATASET	S(8)	f
RANDOM	5,03x	8%
NEARLY SORTED	3,92x	148%
REVERSE	3,61x	174%
DUPLICATES	3,64x	171%



Waktu Eksekusi vs Karakteristik Dataset



- Diurutkan dari dataset paling ringan ke paling berat
- Makin banyak thread → batang makin pendek ✓
- Semua batang di bawah garis merah → paralel lebih cepat ✓

Apa yang Berhasil dan Tidak?



Berhasil

- Merge Sort speedup 3.61-5.03x ✓
- Schedule static/guided efisien ✓
- Superlinear speedup terdeteksi ✓



Tidak Berhasil

- Bitonic Sort paralel lebih lambat ✗
- Dynamic schedule 6-10x lebih lambat ✗

Kesimpulan

- Merge Sort paralel → speedup 5.03x (8 thread)
- Bitonic Sort → tidak efisien (0.37-0.70x)
- Dynamic schedule → fatal untuk Bitonic Sort
- Amdahl's Law → terbukti akurat (selisih 0.03x)
- Konfigurasi optimal → Merge Sort, 4-8 thread



Thank you!

Do you have any questions?